AKADEMIA LEONA KOŹMIŃSKIEGO · INFORMATYKA, I ROK · 16.12.2026

Lab 11

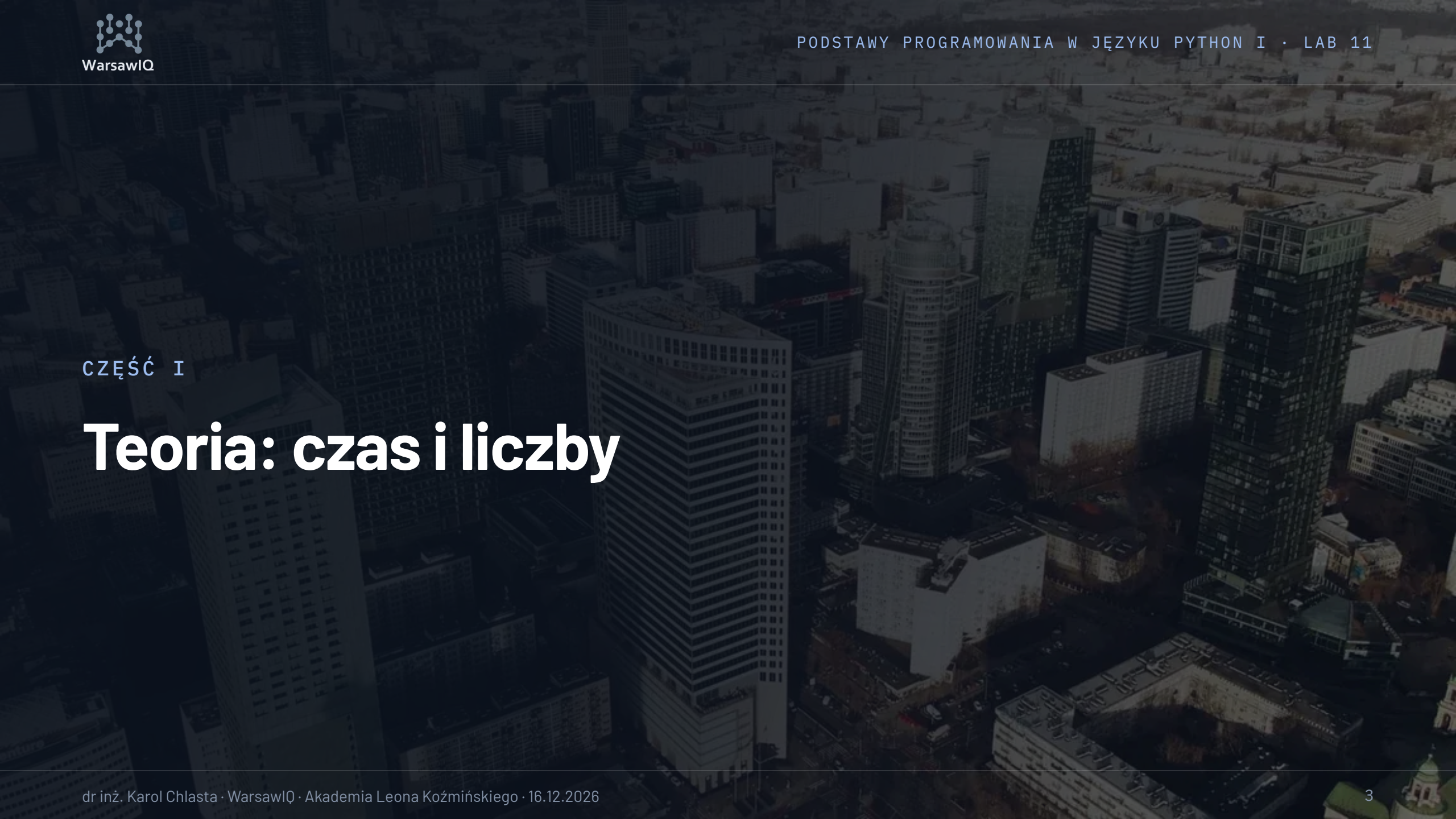
Biblioteka standardowa: daty, czas, matematyka

dr inż. Karol Chlasta · WarsawIQ

Repozytorium kursu: github.com/KarolChlasta/KU-PyDev

Cel zajęć

- Rozumieć, **jak komputer liczy czas** i dlaczego strefy czasowe są trudne
- `datetime`, `timedelta`, formatowanie i parsowanie dat
- Strefy czasowe z `zoneinfo`, czas letni
- Pomiar czasu wykonania; statystyka z `math` i `statistics`
- Efekty z sylabusa: **U2** (biblioteka standardowa), **U1**, **KS1**



CZĘŚĆ I

Teoria: czas i liczby

Czas w komputerze

- **Czas uniksowy**: liczba sekund od **1.01.1970, 00:00:00 UTC** (*epoka*)

```
import time
time.time()          # np. 1797926400.123 – sekundy od epoki
```

- **UTC** (uniwersalny czas koordynowany) – wspólny punkt odniesienia dla całego świata
- Czas lokalny = UTC + **przesunięcie strefy** (w Polsce: +1 h zimą, +2 h latem)

> **Zasada:** przechowuj i licz w **UTC**, zamieniaj na czas lokalny dopiero przy wyświetlaniu.

Moduł `datetime`

TYP	PRZECHOWUJE	PRZYKŁAD
`date`	dzień	`date(2026, 12, 22)`
`time`	godzinę	`time(8, 30)`
`datetime`	dzień + godzinę	`datetime(2026, 12, 22, 8, 30)`
`timedelta`	**różnicę** czasu	`timedelta(days=2, hours=3)`

```
from datetime import date, timedelta
wigilia = date(2026, 12, 24)
(wigilia - date(2026, 12, 22)).days    # 2
date(2026, 12, 22) + timedelta(days=35) # date(2027, 1, 26)
```

Tekst ↔ data

KOD	ZNACZENIE	PRZYKŁAD
`%Y`	rok	2026
`%m`	miesiąc (01–12)	12
`%d`	dzień (01–31)	22
`%H:%M:%S`	godzina:minuty:sekundy	08:30:00
`%A`	nazwa dnia (wg ustawień systemu)	Tuesday

```
from datetime import datetime
datetime.strptime("24.12.2026", "%d.%m.%Y")      # tekst → data („parse”)
datetime(2026, 12, 24).strftime("%Y-%m-%d")      # data → tekst („format”) – '2026-12-24'
date(2026, 12, 24).isoformat()                   # '2026-12-24' – norma ISO 8601
```

Strefy czasowe

```
from datetime import datetime
from zoneinfo import ZoneInfo

meeting = datetime(2026, 12, 22, 10, 0, tzinfo=ZoneInfo("Europe/Warsaw"))
meeting.astimezone(ZoneInfo("America/New_York"))   # 04:00 tego samego dnia
meeting.astimezone(ZoneInfo("Asia/Tokyo"))         # 18:00
```

- Data **naiwna** – bez strefy; **świadoma** (*aware*) – ze strefą. Nie mieszaj ich w obliczeniach
- Nazwy stref pochodzą z bazy **IANA tz** (`Europe/Warsaw`, nie „CET”) – zawiera historię zmian czasu w każdym kraju
- Czas letni (DST) zmienia przesunięcie w ciągu roku – dlatego **nie** licz stref „na sztywno” (+1 h)

Pomiar czasu wykonania

FUNKCJA	DO CZEGO
`time.time()`	aktualny czas (może skoczyć przy zmianie zegara!)
`time.perf_counter()`	**pomiar odcinków czasu** – najdokładniejszy
`time.sleep(s)`	wstrzymanie programu
moduł `timeit`	wielokrotny pomiar krótkich fragmentów kodu

```
start = time.perf_counter()
result = sum_loop(1_000_000)
print(f"{time.perf_counter() - start:.4f} s")
```

- Pojedynczy pomiar bywa zaszumiony – powtarzaj i porównuj rzędy wielkości

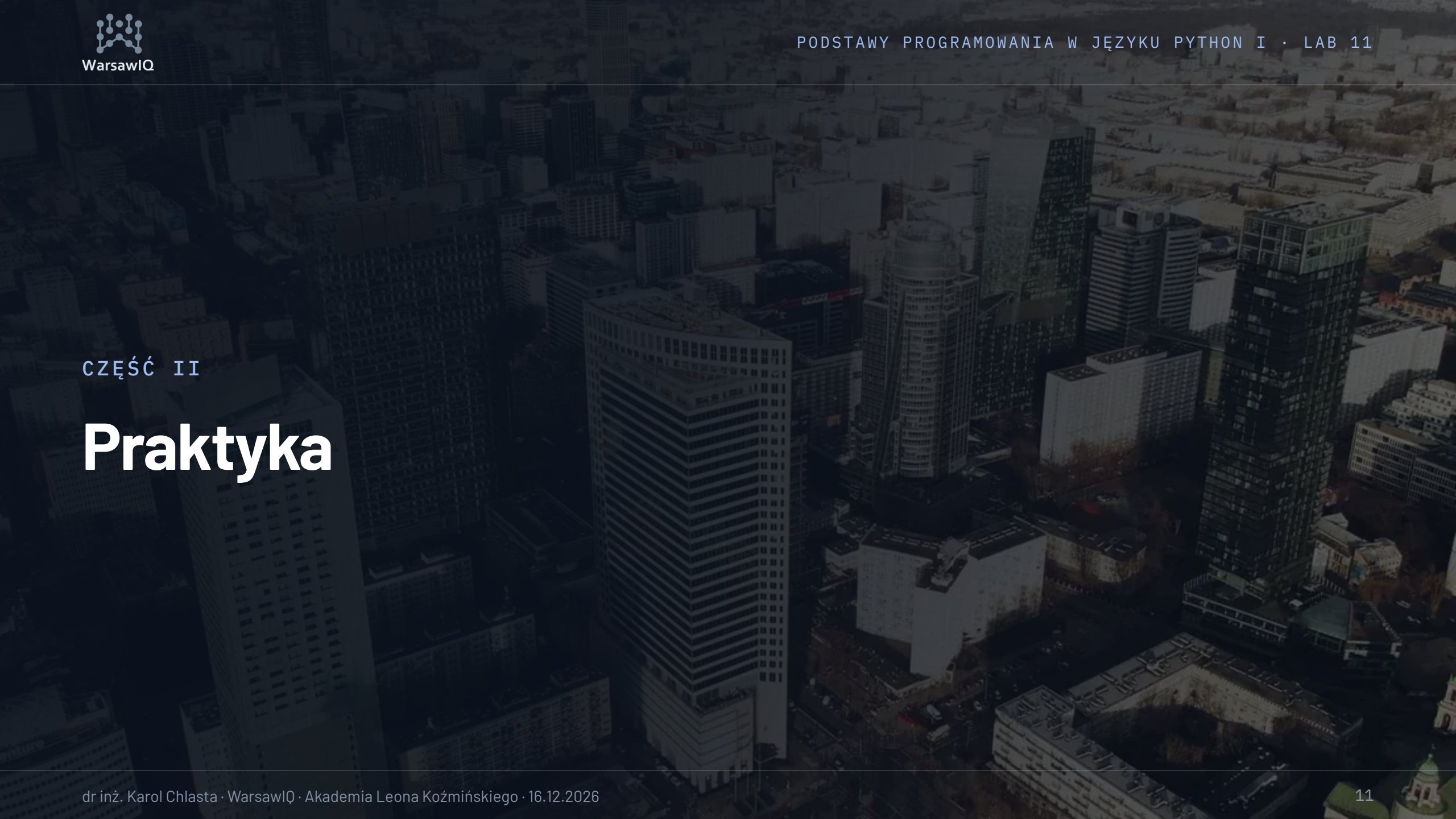
Moduł `math`

FUNKCJA	WYNIK
`math.sqrt(2)`	1.4142135623730951
`math.pi`, `math.e`	stałe
`math.floor(2.7)`, `math.ceil(2.1)`	2, 3
`math.isclose(0.1 + 0.2, 0.3)`	True
pętla `total += 0.1` (10 razy)	0.9999999999999999
`math.fsum([0.1] * 10)`	1.0 – dokładne sumowanie (od Pythona 3.12 także `sum()`)
`math.factorial(5)`, `math.gcd(12, 18)`	120, 6

Statystyka opisowa

MIARA	WZÓR / IDEA	WRAŻLIWA NA WARTOŚCI ODSTAJĄCE?
średnia	$\Sigma x / n$	**tak**
mediana	środkowa wartość po sortowaniu	nie
odchylenie standardowe	$\sqrt{\Sigma(x - \bar{x})^2 / (n - 1)}$	tak

- Dzielnik **n – 1** (próba) zamiast **n** (populacja) – poprawka Bessela; `statistics.stdev` vs `statistics.pstdev`
- Przykład: pensje [4, 5, 5, 6, 100] tys. zł → średnia **24**, mediana **5**. Która lepiej opisuje „typową” pensję?



CZĘŚĆ II

Praktyka

Testowalne funkcje z czasem

```
def days_until(event, today):        # „dziś” jako PARAMETR
    return (event - today).days

days_until(date(2026, 12, 24), date(2026, 12, 22))   # 2 – zawsze, każdego dnia
```

- Funkcja wołająca w środku `date.today()` daje **inny wynik każdego dnia** – nie da się jej porządnie przetestować
- Przekazanie zależności jako parametru (*dependency injection*) wróci w labie 14 przy testach

Ciekawostki ⏰

- **Problem roku 2038**: 32-bitowy licznik sekund od 1970 r. przepełni się 19.01.2038 o 03:14:07 UTC – stare systemy „cofną się" do 1901 r.
- **Sekunda przestępna**: ostatnią dodano 31.12.2016; w 2022 r. Generalna Konferencja Miar zdecydowała o ich zniesieniu do 2035 r.
- Nie wszystkie strefy różnią się o pełne godziny: Indie mają UTC+5:30, a **Nepal UTC+5:45**
- Według anegdoty mały **Gauss** zsumował liczby 1–100 w kilka chwil, łącząc je w pary 1+100, 2+99… – stąd wzór n(n+1)/2 w `zad4_pomiar.py`

Dzisiejsze zadania

	PLIK	ZADANIE
★	`zad1_wydarzenia.py`	przypomnienie o wydarzeniach
★	`zad2_strefy.py`	strefy czasowe
★	`zad3_statystyka.py`	średnia, mediana, odchylenie
★	`zad4_pomiar.py`	pomiar czasu
☆	`zad5_kalendarz.py`	dni robocze

Sprawdzenie i oddanie

```
cd lab11_daty_math
python -m unittest discover -s tests -v
git add .
git commit -m "Lab 11"
git push
```

- Dziś wyjątkowo **środa 14:00–17:15**
- Punkt aktywności: **obecność + wszystkie testy ★ zielone + push na GitHub przed 17:15**
- Windows: `python -m pip install tzdata`, jeśli `ZoneInfoNotFoundError`

Do poczytania

- Dokumentacja: `datetime`, `zoneinfo`, `time`, `timeit`, `math`, `statistics`
- *Falsehoods programmers believe about time* – lista błędnych założeń programistów o czasie
- PEP 615 – *Support for the IANA Time Zone Database in the Standard Library*

Następny lab (22.12): obsługa wyjątków